

사용자 활동 시간대 분석

[38 반 / 최민혁 / 5694]

작성일 2026 년 7 월 14 일

Q1. 차트로 표현하기

이 파일은 pycparser 로 만든 C 언어 AST 를 JSON 으로 저장한 파일이다. 가장 위에는 FileAST 가 있고, 그 안의 ext 배열에 전역변수와 함수가 순서대로 들어 있다. ext 에는 모두 55 개의 항목이 있었고, FuncDef 는 36 개, Decl 은 19 개였다.

1.1 주요 필드의 의미

_nodetype: 현재 객체가 어떤 AST 문법 노드인지 나타낸다. 예를 들어 FuncDef 는 함수 정의, If 는 조건문, BinaryOp 는 이항 연산이다.

coord: 원본 소스의 파일명, 행, 열 위치이다. target.c:94:5 는 target.c 의 94 행 5 열을 의미한다.

decl / type: decl 은 선언 전체를, type 은 TypeDecl, PtrDecl, IdentifierType 등으로 연결되는 타입 구조를 나타낸다.

body / block_items: 함수나 중괄호 블록의 내부 문장 목록이다.

cond / iftrue / iffals: if 조건식과 참/거짓 분기를 나타낸다. else if 는 iffals 안에 또 다른 If 노드가 들어가는 방식으로 표현된다.

1.2 Decl, FuncDecl, FuncDef 구분

Decl 은 변수나 함수 등을 선언한 노드이다. FuncDecl 은 함수의 반환형과 파라미터 같은 함수의 모양을 나타낸다. FuncDef 는 함수 몸체까지 들어 있는 실제 함수이다. 예를 들어 exit 은 함수 원형이라 ext[0]에 Decl 로 있고, main 은 body 가 있기 때문에 ext[5]에 FuncDef 로 있다.

FuncDecl 을 전부 세면 실제 함수 안에 들어 있는 FuncDecl 까지 같이 세어서 42 개가 나온다. 그래서 함수 개수는 ext 에서 FuncDef 만 세었다.

1.3 ext 배열과 함수 개수

json_len(ext)를 사용하면 현재 배열의 길이를 알 수 있다. json_c.c 에서는 마지막 인덱스에 1 을 더해서 길이를 구한다. 이 파일의 마지막 인덱스는 54 이므로 길이는 55 가 된다. main 에서는 0 번부터 54 번까지 확인하면서 _nodetype 이 FuncDef 인 것만 세어 함수는 36 개, 나머지 19 개는 Decl 인 걸 알았다.

2. 분석기 설계 및 구현

2.1 분석기 전체 소스코드

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "json_c.c"

char *read_file(char *filename)
{
    FILE *fp;
    long size;
    char *text;

    fp = fopen(filename, "rb");
    if (fp == NULL) {
        printf("파일을 열 수 없습니다.\n");
        return NULL;
    }

    fseek(fp, 0, SEEK_END);
    size = ftell(fp);
    rewind(fp);

    text = (char *)malloc(size + 1);
    fread(text, 1, size, fp);
    text[size] = '\0';

    fclose(fp);
    return text;
}

void get_type(json_value node, char *result)
{
    json_value node_type;
    json_value names;
    int i;

    node_type = json_get(node, "_nodetype");

    if (strcmp(json_to_string(node_type), "IdentifierType") == 0) {
        names = json_get(node, "names");

        for (i = 0; i < json_len(names); i++) {
            if (i != 0)
                strcat(result, " ");
            strcat(result, json_to_string(json_get(names, i)));
        }
    }
    else if (strcmp(json_to_string(node_type), "PtrDecl") == 0) {
        get_type(json_get(node, "type"), result);
        strcat(result, " *");
    }
    else if (strcmp(json_to_string(node_type), "ArrayDecl") == 0) {
        get_type(json_get(node, "type"), result);
        strcat(result, "[ ]");
    }
}
```

```

    }
    else {
        get_type(json_get(node, "type"), result);
    }
}

int count_if(json_value node)
{
    int count = 0;
    int i;

    if (node.type == JSON_OBJECT) {
        json_object *obj = (json_object *)node.value;
        json_value node_type = json_get(node, "_nodetype");

        if (node_type.type == JSON_STRING &&
            strcmp(json_to_string(node_type), "If") == 0)
            count++;

        for (i = 0; i <= obj->last_index; i++)
            count += count_if(obj->values[i]);
    }
    else if (node.type == JSON_ARRAY) {
        json_array *arr = (json_array *)node.value;

        for (i = 0; i <= arr->last_index; i++)
            count += count_if(arr->values[i]);
    }

    return count;
}

int main(void)
{
    char *text;
    json_value root;
    json_value ext;
    int function_count = 0;
    int number = 0;
    int i;

    text = read_file("ast.json");
    if (text == NULL)
        return 1;

    root = json_create(text);
    free(text);

    ext = json_get(root, "ext");

    for (i = 0; i < json_len(ext); i++) {
        json_value item = json_get(ext, i);
        json_value node_type = json_get(item, "_nodetype");

        if (strcmp(json_to_string(node_type), "FuncDef") == 0)
            function_count++;
    }

    printf("전체 함수 개수: %d\n\n", function_count);
}

```

```

for (i = 0; i < json_len(ext); i++) {
    json_value function = json_get(ext, i);
    json_value node_type = json_get(function, "_nodetype");
    json_value decl;
    json_value function_type;
    json_value args;
    json_value params;
    char return_type[100] = "";
    int j;

    if (strcmp(json_to_string(node_type), "FuncDef") != 0)
        continue;

    number++;
    decl = json_get(function, "decl");
    function_type = json_get(decl, "type");

    get_type(json_get(function_type, "type"), return_type);

    printf("%02d. %s %s(",
        number,
        return_type,
        json_to_string(json_get(decl, "name")));

    args = json_get(function_type, "args");

    if (args.type != JSON_NULL) {
        params = json_get(args, "params");

        for (j = 0; j < json_len(params); j++) {
            json_value param = json_get(params, j);
            json_value param_name = json_get(param, "name");
            char param_type[100] = "";

            if (j != 0)
                printf(", ");

            get_type(json_get(param, "type"), param_type);
            printf("%s", param_type);

            if (param_name.type == JSON_STRING)
                printf(" %s", json_to_string(param_name));
        }

        printf(") if=%d\n",
            count_if(json_get(function, "body")));
    }

    json_free(root);
    return 0;
}

```

2.2 get_type 함수

get_type 함수는 함수의 반환형과 파라미터 타입을 문자열로 만드는 함수이다. AST의 타입 정보는 TypeDecl, PtrDecl, IdentifierType 처럼 여러 노드 안에 중첩되어 있고, 타입마다 중첩 깊이도 다를

수 있다. type 을 재귀적으로 따라가도록 구현했다. node 에는 현재 확인할 타입 노드가 들어오고, result 에는 완성한 타입 문자열을 저장한다.

IdentifierType 을 만나면 실제 타입 이름이 들어 있는 names 배열을 읽는다. names 에 int 나 char 같은 문자열이 들어 있고, 여러 단어가 있으면 중간에 공백을 넣어 result 에 붙인다. PtrDecl 을 만나면 내부 type 을 먼저 get_type 으로 확인한 뒤 별표를 붙인다. ArrayDecl 도 내부 타입을 먼저 확인한 뒤 대괄호를 붙인다. TypeDecl 이나 FuncDecl 같은 중간 노드는 별도의 글자를 붙이지 않고 안쪽 type 으로 계속 들어간다.

char *를 예로 들면 첫 번째 호출에서는 PtrDecl 을 확인한다. PtrDecl 은 바로 별표를 붙이지 않고 먼저 TypeDecl 을 대상으로 get_type 을 다시 호출한다. TypeDecl 도 안쪽의 IdentifierType 을 대상으로 get_type 을 호출한다. IdentifierType 에서는 names 의 char 를 result 에 넣으므로 이때 result 는 char 가 된다. 호출이 TypeDecl 로 돌아오면 문자를 붙이지 않고 그대로 복귀한다. 마지막으로 PtrDecl 까지 돌아오면 result 뒤에 별표를 붙여 char *를 완성한다. 이런 순서 때문에 포인터의 기본 타입이 먼저 출력된다.

2.3 count_if 함수

count_if 함수는 전달받은 노드 아래에 있는 if 노드의 개수를 세어 반환한다. 함수 body 는 객체와 배열이 여러 단계로 중첩된 트리이고, if 가 어느 깊이에 들어 있을지 미리 알 수 없기 때문에 모든 자식 노드를 같은 방법으로 확인할 수 있도록 재귀 함수로 만들었다. 호출될 때마다 count 를 0 으로 시작하고, 현재 값이 JSON_OBJECT 이면 _nodetype 을 확인한다. 그 값이 if 이면 현재 노드 한 개를 찾았으므로 count 를 1 증가시킨다.

현재 객체만 확인하고 끝내면 중첩된 if 를 찾을 수 없으므로 안에 들어있는 모든 value 를 다시 count_if 에 전달한다. 각 재귀 호출이 반환한 값을 현재 count 에 더한다. 현재 값이 JSON_ARRAY 이면 배열의 0 번부터 last_index 까지 같은 방법으로 확인한다. 문자열, 숫자, null 처럼 객체나 배열이 아닌 값은 안쪽에 다른 노드가 없으므로 0 을 반환한다.

main 에서는 함수 전체가 아니라 function 의 body 만 count_if 에 전달한다. 따라서 현재 함수 본문 안의 if 만 계산된다. if 안에 다른 if 가 있으면 재귀 호출을 통해 각각 한 개씩 세고, else if 도 AST 에서는 별도의 if 노드이므로 한 개로 센다.

실제 takechar 함수의 body 를 예로 들면 처음에는 Compound 노드를 확인하므로 count 는 0 이다. Compound 의 block_items 배열로 들어가 첫 번째 원소인 if 를 만나면 count 가 1 이 된다. 이 if 의

cond 와 iftrue 안쪽도 다시 확인하지만 추가 if 는 없다. 배열에 남은 세 원소도 모두 Assignment 라서 count 가 증가하지 않는다. 모든 재귀 호출이 끝나면 takechar 의 최종 결과로 1 을 반환한다.

2.4 main 함수의 흐름

파일 읽기, JSON 파싱, ext 배열 가져오기, 함수 개수 계산, 함수별 정보 출력, JSON 메모리 해제 순서로 실행된다.

먼저 read_file 로 ast.json 의 내용을 문자열로 읽고 json_create 로 JSON 트리를 만든다. 그다음 root 에서 ext 배열을 가져온다. 첫 번째 반복문에서는 ext 를 돌면서 FuncDef 의 개수를 세고, 두 번째 반복문에서는 각 FuncDef 의 이름, 반환형, 파라미터, if 개수를 출력한다. 모든 출력이 끝나면 json_free 로 JSON 트리의 메모리를 해제한다.

2.5 파일 읽기와 JSON 생성

```
int main(void)
{
    char *text;
    json_value root;
    json_value ext;
    int function_count = 0;
    int number = 0;
    int i;

    text = read_file("ast.json");
    if (text == NULL)
        return 1;
```

json_create 는 JSON 문자열을 받아서 파싱하는 함수이다. 하지만 ast.json 은 파일이므로 먼저 파일 내용을 문자열로 읽어야 했다. 그래서 read_file 함수를 만들었다. fseek 와 ftell 로 파일 크기를 구하고, malloc 으로 크기보다 1 바이트 크게 메모리를 잡았다. fread 로 내용을 읽은 뒤 마지막에 널 문자를 넣고 json_create 로 넘겼다. 파일을 열지 못하면 NULL 을 반환하도록 했다.

2.6 ext 배열에서 함수 찾기

```
for (i = 0; i < json_len(ext); i++) {
    json_value item = json_get(ext, i);
    json_value node_type = json_get(item, "_nodetype");

    if (strcmp(json_to_string(node_type), "FuncDef") == 0)
        function_count++;
}
```

main 에서 json_get(root, "ext")를 사용해 ext 배열을 가져왔다. 첫 번째 반복문에서는 각 원소의 _nodetype 을 확인하고 FuncDef 일 때만 function_count 를 1 씩 늘렸다. 두 번째 반복문에서는 FuncDef 가 아닌 원소를 continue 로 넘겼다. 이렇게 해서 함수 원형이나 전역변수는 빼고 실제 함수만 출력했다.

2.7 함수 이름과 반환형 찾기

```
for (i = 0; i < json_len(ext); i++) {
    json_value function = json_get(ext, i);
    json_value node_type = json_get(function, "_nodetype");
    json_value decl;
    json_value function_type;
    json_value args;
    json_value params;
    char return_type[100] = "";
    int j;

    if (strcmp(json_to_string(node_type), "FuncDef") != 0)
        continue;

    number++;
    decl = json_get(function, "decl");
    function_type = json_get(decl, "type");

    get_type(json_get(function_type, "type"), return_type);

    printf("%02d. %s %s(",
        number,
        return_type,
        json_to_string(json_get(decl, "name")));
}
```

```

}
else if (strcmp(json_to_string(node_type), "ArrayDecl") == 0) {
    get_type(json_get(node, "type"), result);
    strcat(result, "[");
}
else {
    get_type(json_get(node, "type"), result);
}
}

```

함수 이름은 decl 안의 name 에서 가져왔다. 반환형은 type 안에 여러 번 들어가 있어서 get_type 함수로 따로 처리했다. TypeDecl 이나 FuncDecl 을 만나면 그 안의 type 으로 계속 들어간다. IdentifierType 을 만나면 int, void, char 같은 실제 타입을 얻는다. PtrDecl 이면 별표를 붙이고 ArrayDecl 이면 대괄호를 붙였다.

2.8 파라미터 찾기

```

args = json_get(function_type, "args");

if (args.type != JSON_NULL) {
    params = json_get(args, "params");

    for (j = 0; j < json_len(params); j++) {
        json_value param = json_get(params, j);
        json_value param_name = json_get(param, "name");
        char param_type[100] = "";

        if (j != 0)
            printf(", ");

        get_type(json_get(param, "type"), param_type);
        printf("%s", param_type);

        if (param_name.type == JSON_STRING)
            printf(" %s", json_to_string(param_name));
    }
}

```

args 가 null 이 아니면 params 배열을 반복문으로 돌리고, 각 파라미터의 type 은 get_type 으로 구하고 name 은 변수명으로 출력했다. args 가 null 이면 파라미터가 없는 함수이므로 괄호 안에 아무것도 출력하지 않았다.

2.9 if 개수 세기

```
printf("    if=%d\n",  
       count_if(json_get(function, "body")));
```

마지막으로 각 함수의 body 를 count_if 함수에 넘겨 if 개수를 구했다. body 만 전달했기 때문에 현재 함수 안에 있는 if 노드만 계산된다. count_if 가 반환한 값은 함수 정보 뒤에 함께 출력했다.

3. 실행 결과

3.1 결과 요약

프로그램을 실행해 보니 실제 함수는 모두 36 개였다. 반환형은 void 가 18 개, int 가 17 개, char *가 1 개였다. 각 함수에서 찾은 if 를 모두 더하면 54 개였다.

처음에 FuncDecl 을 세어 보니 42 개가 나왔다. 하지만 여기에는 함수 원형과 실제 함수 안의 FuncDecl 이 같이 들어 있었다. ext 에서 FuncDef 만 세었을 때 실제 함수 36 개를 얻을 수 있었다.

3.2 전체 함수 분석 결과

아래 결과는 분석기 실행 결과이다.

```

choi@choi:~/whs4/programming$ ./ast_analyzer
전체 함수 개수 : 36

01. int main() if=0
02. char * my_realloc(char * old, int oldlen, int newlen) if=0
03. void error() if=0
04. void takechar() if=1
05. void get_token() if=7
06. int peek(char * s) if=0
07. int accept(char * s) if=1
08. void expect(char * s) if=1
09. void save_int(char * p, int n) if=0
10. int load_int(char * p) if=0
11. void emit(int n, char * s) if=1
12. void be_push() if=0
13. void be_pop(int n) if=0
14. int sym_lookup(char * s) if=1
15. void sym_declare(char * s, int type, int value) if=1
16. int sym_declare_global(char * s) if=1
17. void sym_define_global(int current_symbol) if=1
18. void sym_get_value(char * s) if=5
19. void be_start() if=0
20. void be_finish() if=0
21. void promote(int type) if=2
22. int primary_expr() if=9
23. void binary1(int type) if=0
24. int binary2(int type, int n, char * s) if=0
25. int postfix_expr() if=3
26. int additive_expr() if=2
27. int shift_expr() if=2
28. int relational_expr() if=0
29. int equality_expr() if=2
30. int bitwise_and_expr() if=0
31. int bitwise_or_expr() if=0
32. int expression() if=2
33. void type_name() if=0
34. void statement() if=8
35. void program() if=4
36. int main1() if=0

```

4. 원본 소스코드 복원

4.1 복원 방법

ast.json 에서 함수 정의는 ext 배열 안의 FuncDef 로 구분된다. FuncDef 의 decl 에는 함수 이름, 반환형, 파라미터가 있고, body 에는 함수 내부의 문장이 저장되어 있다. body.block_items 는 문장들이 실행되는 순서대로 나열되어 있으므로, decl 로 함수 선언부를 만들고 block_items 를 차례대로 읽어 함수 내부를 작성했다.

각 문장은 _nodetype 을 기준으로 C 문법에 맞게 옮겼다. Decl 은 변수 선언, Assignment 는 대입문, FuncCall 은 함수 호출을 뜻한다. BinaryOp 는 left, op, right 를 조합해 연산식을 만들었고, If 와

While 은 cond 에 저장된 조건과 내부 문장을 이용했다. malloc 과 getchar 는 FuncCall 이지만 #include 는 AST 에 저장되어 있지 않아서, stdlib.h 와 stdio.h 를 추가했다.

4.2 my_realloc 함수 복원

```
char *my_realloc(char *old, int oldlen, int newlen)
{
    char *new = malloc(newlen);
    int i = 0;

    while (i <= oldlen - 1)
    {
        new[i] = old[i];
        i = i + 1;
    }

    return new;
}
```

my_realloc 의 decl.type 은 FuncDecl 이고 반환형 부분은 PtrDecl 로 되어 있다. 그 안의 IdentifierType 이 char 이므로 반환형은 char *가 된다. args.params 에는 char *old, int oldlen, int newlen 에 해당하는 세 개의 Decl 이 들어 있어 함수 선언부를 만들 수 있었다.

body.block_items 의 처음 두 노드는 변수 선언이다. 첫 번째 Decl 은 char *new 이며, 초기값이 malloc 을 호출하는 FuncCall 로 되어 있다. 이 함수 호출의 인자가 newlen 이므로 char *new = malloc(newlen);으로 작성했다. 다음 Decl 은 int 형 변수 i 이고 초기값이 Constant 0 이어서 int i = 0;이 된다.

세 번째 노드는 While 이다. 조건의 바깥쪽 연산자는 <=이고 왼쪽에는 i, 오른쪽에는 oldlen - 1 이 들어 있다. While 내부에는 두 개의 Assignment 가 있으며, 첫 번째는 ArrayRef 를 사용한 new[i] = old[i];이고 두 번째는 i = i + 1;이다. 이 구조를 보면 기존 배열의 값을 새로운 배열로 하나씩 복사하는 반복문이라는 것을 알 수 있다.

마지막 Return 노드는 new 를 반환한다. 이를 통해 이 함수는 새로운 메모리를 할당하고 기존 내용을 복사한 뒤 그 주소를 리턴하는 함수라는 것을 확인할 수 있다.

4.3 takechar 함수 복원

```
void takechar()
{
    if (token_size <= i + 1)
    {
        int x = (i + 10) << 1;
        token = my_realloc(token, token_size, x);
        token_size = x;
    }

    token[i] = nextc;
    i = i + 1;
    nextc = getchar();
}
```

takechar의 반환형은 IdentifierType의 void이고 args가 null이므로 파라미터가 없는 함수이다. body의 첫 번째 노드는 If이며, 조건은 바깥쪽 BinaryOp의 <=와 그 안의 i + 1을 조합하면 token_size <= i + 1이 된다.

iftrue안의 첫 번째 Decl은 변수 x이다. 초기값은 i + 10을 먼저 계산하고 그 결과를 1만큼 왼쪽으로 이동하는 구조라서 int x = (i + 10) << 1;로 옮겼다. 다음 Assignment에서는 my_realloc을 호출하고 있다. ExprList에 token, token_size, x가 순서대로 들어 있으므로 세 값을 함수의 인자로 사용했다. 그다음에는 새 크기인 x를 token_size에 저장한다.

If문 뒤의 세 개의 Assignment는 현재 문자를 처리하는 부분이다. token[i] = nextc;로 문자를 배열에 넣고, i = i + 1;로 다음 위치로 이동한다. 마지막에는 인자가 없는 getchar를 호출해 다음 문자를 nextc에 저장한다.

token, token_size, nextc, i는 takechar 내부에 Decl이 없다. 대신 FileAST.ext에서 함수와 같은 단계의 Decl로 선언되어 있으므로 전역변수를 사용하는 것으로 판단했다.

4.4 peek 함수 복원

```
int peek(char *s)
{
    int i = 0;

    while ((s[i] == token[i]) & (s[i] != 0))
    {
        i = i + 1;
    }

    return s[i] == token[i];
}
```

peek 는 int 를 반환하며 args.params 에는 파라미터가 한 개 있다. 파라미터의 구조가 PtrDecl 안의 char 이고 이름이 s 이므로 함수 선언부는 int peek(char *s)가 된다. body 의 첫 번째 Decl 은 지역변수 i 를 0 으로 초기화하는 부분이다.

While 의 조건은 가장 바깥쪽 BinaryOp 의 연산자가 &이고, 왼쪽에는 s[i] == token[i], 오른쪽에는 s[i] != 0 이 들어 있다. s[i]와 token[i]는 모두 ArrayRef 이며 name 은 배열 이름, subscript 는 인덱스를 나타낸다. 이를 조합하면 조건은 (s[i] == token[i]) & (s[i] != 0)이 된다. 일반적으로는 &&도 생각할 수 있지만 JSON 의 op 값이 &이므로 그대로 사용했다.

While 내부에서는 i 를 1 씩 증가시킨다. 반복문이 끝난 뒤의 Return 에는 s[i] == token[i]가 저장되어 있어 두 문자가 같은지를 반환하도록 작성했다. 전체 구조를 보면 전달받은 문자열과 현재 token 의 내용을 앞에서부터 비교하는 함수라고 볼 수 있다.

5. 결론 및 질문

5.1 수행 결과 및 느낀 점

처음 ast.json 을 열었을 때는 줄 수가 10,000 줄이 넘어 함수 이름과 반환형을 어디서 찾아야 하는지 잘 몰랐다. 확인해 보니 Decl, FuncDecl, TypeDecl, IdentifierType 안쪽으로 계속 들어가야 했다. 직접 코드를 작성하면서 AST 가 C 코드를 여러 노드로 나누어 저장한다는 것을 알 수 있었다. 중첩된 타입이나 if 를 찾을 때 재귀 함수가 필요한 이유도 이해할 수 있었다.

처음에는 FuncDecl 이 함수라고 생각해서 42 개가 나왔다. 하지만 FuncDecl 에는 함수 원형과 실제 함수 안의 타입 정보도 들어 있었다. ext 에서 FuncDef 만 확인해야 실제 함수 36 개가 나온다는 것을 알게 되었다.

5.2 질문

ast.json 을 확인하면서 AST 에는 함수와 연산 구조는 남아 있지만, #include 나 #define 같은 전처리문과 연산 순서에 영향을 주지 않는 괄호는 저장되지 않는다는 것을 알았습니다. 그럼 이번 과제와 같이 AST 로부터 소스코드를 복원할 때 이런 전처리문들은 어떻게 다시 작성해야 하는지 궁금합니다.